

Relatório de Evolução — Sprint 03

Projeto GoodWe Charge Assistant — EV Challenge

1. Resumo da Evolução

Nas Sprints 1 e 2, o grupo desenvolveu um chatbot conversacional para suporte a carregadores elétricos e eletropostos GoodWe, utilizando chamadas diretas a um modelo de linguagem (OpenAI GPT-4o-mini) dentro de um loop manual (*while True*), com contexto definido via system prompt fixo e histórico de conversa controlado manualmente pelo próprio código, sem uso de um framework de orquestração de agentes.

Na Sprint 03, o núcleo conversacional foi refatorado para utilizar o **LangGraph**, framework de orquestração de agentes do ecossistema LangChain. A refatoração incorporou: memória de sessão gerenciada nativamente pelo framework (via *checkpoint*), guardrails de segurança e escopo definidos por *system prompt* estruturado, testes formais de resistência a Prompt Injection, e comparação sistemática entre dois modelos de linguagem de provedores distintos (Groq e Google Gemini). A finalidade original do projeto — suporte a usuários de carregadores GoodWe — foi mantida integralmente.

2. Refatoração — Decisões Técnicas e Trade-offs

Framework escolhido: LangGraph, por já haver familiaridade prévia com o ecossistema LangChain (utilizado na Sprint 2) e por oferecer gerenciamento de estado e memória de sessão nativos, resolvendo diretamente o requisito de memória por sessão sem necessidade de implementação manual de histórico.

Componentes utilizados: *StateGraph* e *MessagesState* para orquestração do fluxo conversacional; *MemorySaver* como checkpointer de memória por sessão (identificada por *thread_id*); *SystemMessage* como guardrail de comportamento e escopo; *init_chat_model* como camada de abstração que permite alternar entre provedores de modelo sem alterar a lógica do agente.

Trade-offs identificados: o *MemorySaver* padrão armazena o histórico apenas em memória RAM do processo, sem persistência entre execuções — solução aceitável para o escopo da sprint, mas que exigiria substituição por um checkpointer persistente (ex.: baseado em SQLite ou banco de dados) em um cenário de produção real. Além disso, observou-se que o uso de guardrail via system prompt reduz, mas não elimina, comportamentos indesejados do modelo (detalhado na Seção 4).

3. Comparativo Antes x Depois

Critério	Sprints 1 e 2 (legado)	Sprint 03 (atual)
Arquitetura	Loop manual (<i>while True</i>) com chamadas diretas à API	Grafo de estados orquestrado pelo LangGraph
Memória	Controlada manualmente no código da aplicação	Gerenciada nativamente pelo framework (<i>checkpoint</i>)
Modelo utilizado	OpenAI GPT-4o-mini (único modelo, sem comparação)	Groq (GPT-OSS 120B) selecionado após comparação com Gemini 3.6 Flash
Segurança/Guardrails	System prompt único, sem testes formais	System prompt reforçado + 2 testes de Prompt Injection documentados
Resultados de segurança	Não avaliados formalmente	Resistência confirmada a ataque direto e a variação disfarçada, nos 2 modelos
Comportamento observado	Alucinação técnica não identificada/documentada	Alucinação técnica identificada, mitigada (não eliminada) via guardrail

Resposta à pergunta central: sim, a nova arquitetura tornou o chatbot mensuravelmente mais robusto. A migração para o LangGraph eliminou a necessidade de controle manual de histórico, a comparação entre modelos permitiu uma escolha justificada por evidência (e não por preferência), e os testes de segurança formais expuseram comportamentos (alucinação técnica) que passavam despercebidos na versão anterior, permitindo mitigação ativa via guardrail.

4. Problemas Encontrados e Soluções

Problema 1: Obsolescência rápida de identificadores de modelo

Problema: durante o desenvolvimento, os identificadores de modelo utilizados inicialmente (*llama-3.3-70b-versatile* no Groq e *gemini-2.5-flash* no Gemini) retornaram erro HTTP 404, indicando descontinuação pelos provedores. **Alternativas possíveis:** (a) fixar a aplicação a uma versão de modelo mais antiga e potencialmente instável; (b) consultar manualmente a documentação de cada provedor a cada erro. **Solução adotada:** implementação de um script auxiliar que consulta a API de cada provedor e lista os modelos disponíveis para a chave em uso, permitindo atualização rápida do identificador sem depender de documentação externa desatualizada. **Justificativa:** essa abordagem torna a aplicação resiliente a mudanças futuras no catálogo de modelos, sem exigir intervenção manual extensa.

Problema 2: Limite de cota da camada gratuita do Gemini

Problema: a camada gratuita do modelo Gemini apresentou um limite de 20 requisições por dia, interrompendo os testes com erro HTTP 429 (*RESOURCE_EXHAUSTED*) em mais de uma ocasião durante o desenvolvimento. **Alternativas possíveis:** (a) migrar para um plano pago do Gemini; (b) reduzir a quantidade de testes realizados no modelo; (c) implementar tratamento de exceção específico. **Solução adotada:** implementação de tratamento de exceção (*try/except*) específico para *GoogleRateLimitError*, permitindo que a aplicação registre a falha de forma controlada e continue a execução dos demais testes, em vez de interromper todo o processo. **Justificativa:** essa solução não elimina a limitação de cota (uma característica do provedor, fora do controle do grupo), mas evita que ela comprometa a robustez geral da aplicação, e documenta a limitação como um critério legítimo de comparação entre modelos (Bloco B).

Problema 3: Alucinação de especificações técnicas

Problema: ambos os modelos, ao responderem sobre detalhes do carregamento, geraram especificações técnicas não confirmadas (tabelas de tarifa, modelos de equipamento e recursos fictícios), mesmo sem solicitação explícita do usuário. **Alternativas possíveis:** (a) aceitar o comportamento como limitação inerente ao modelo; (b) adicionar uma camada de validação de saída (nó adicional no grafo); (c) reforçar o guardrail via system prompt. **Solução adotada:** reforço do *system prompt* com instrução explícita de não inventar dados técnicos e de admitir desconhecimento quando aplicável. **Justificativa:** a solução reduziu significativamente a frequência e a extensão da alucinação nos testes subsequentes, embora não a tenha eliminado por completo — a implementação de uma camada de validação de saída foi identificada como evolução futura recomendada, não realizada nesta sprint por restrição de tempo.

5. Divisão da Equipe

Nome	RM	Responsabilidade principal
Renan de Castro Albuquerque	570532	Implementação técnica integral: arquitetura LangGraph, integração com Groq e Gemini, guardrails de segurança, testes de memória e Prompt Injection, documentação técnica
Ângelo Malta Reina	570769	Apoio consultivo ao desenvolvimento
Gustavo Mendonça Duarte	570561	Apoio consultivo ao desenvolvimento
Matheus Carpinheiro Moreno	571770	Apoio consultivo ao desenvolvimento
Vinícius Souza Ferraz	570622	Apoio consultivo ao desenvolvimento

Turma: 1 CCPX. A implementação técnica desta sprint foi conduzida integralmente por Renan de Castro Albuquerque; os demais integrantes contribuíram com apoio consultivo, enquanto se dedicavam a outras frentes do curso no mesmo período.